Week 2

12 October 2026 19:01

Complexity (Time, Space)

Sorting Algos

Insertion Sort

Selection sort

Merge sort

Complexity of default python func. or features

Recursive

```
def gcd(m,n):
    a,b = max(m,n), min(m,n)  # a>b  1
    if a%b == 0:  1 (Best case scenario, this will evaluate to true)
        return b
    else:
        return gcd(b, a%b)
```

Best case: 2 lines executed

gcd(a,b) [a>b]

↓

gcd(a, a%b)

```
def prime_check(n):                          prime ✓                    prime ✗
    for i in range(2, int(√n)+1):  → int(√n)   1≤k<int(√n)
        if n%i == 0:  → int(√n)                1≤k<int(√n)
            return False                       1
    return True → 1                when 'n' is prime    when 'n' is not prime
                                   2(int(√n))+1         2k+1
```

$O(2(int(\sqrt{n}))+1) = O(int(\sqrt{n})) = O(\sqrt{n})$ upperbound

$O \quad \Omega \quad \Theta$

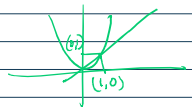
$\Omega \leq \Theta < O$

$2x^2+x+3 \longrightarrow O(2x^2+x+3)$

$3x^2+x+2 \longrightarrow O(3x^2+x+2)$

$1000x^2+10000 \longrightarrow O(1000x^2+10000)$

$= O(x^2)$ Quadratic growing time complexity




$ax^2 + f(x) + const$

$f(x)$ grows slower than $x^2$

```
s=0  1                    i  i  i  i  i
for i in range(5)         0  1  2  3  4  (5)
    for j in range(5)     j=0...4   j=0...4   (5)x5
        s=s+1             (5x5)     5         10        25
print(s)  1
```

$1+n+n^2+n^2+1 \Rightarrow 2n^2+n+2$

list.pop() O(1)

list.pop(3)

1) What is the time complexity of the following code?

```
1  def fun(L, n):
2      if n <= 0:
3          return
4      for i in range(n):
5          print(L[i])
6      fun(L, n - 1)
```

$R(n) = n + R(n-1)$

$R(n) = n + (n-1+R(n-2))$

$R(n) = n + n-1 + n-2 + \cdots + 1 + R(0)$

fun(L,n) → n

fun(L,n-1) → n-1

n-2

⋮

n=0

$S = 1+2+3+\cdots+n-1+n$

$S = n+n-1+\cdots+2+1$

$2S = (n+1)+(n+1)+\cdots+(n+1)$

$S = \frac{n(n+1)}{2}$

$\frac{n(n+1)}{2} \rightarrow O\left(\frac{n^2}{2}+\frac{n}{2}\right) = O(n^2)$

fun(L,n)

```
def fun(L,n):
    if n≤0:
    return
```

fun(L,n)
fun(L,n-1)
fun(L,n-2)
fun(L,0)

Same as in Qs (Iterative)

```
def fun(L,n):
    if n<=0:
        return
01  for i in range(n):
02      for j in range(n-i):
03          print(L[j])
```

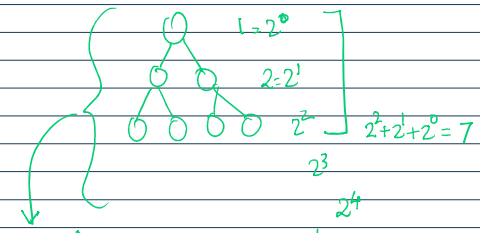
i=0 i=1 - - - i=n-1
n n-1 1
$O(\frac{n(n+1)}{2}) = O(n^2)$
$O(\frac{n^2+n}{2}) = O(\frac{n^2+n}{2})$

```
01 for i in range(5):
02     for j in range(n/2):
03         s=s+1
```

i=0 - - - i=5
n/2 n/2
$O(5n/2) = O(n)$

```
01 for i in range(n):
02     for j in range(n/3):
03         print(i)
```

i=0 - - - i=n
n/3 n/3
$O(n^2/3) = O(n^2)$




$2^2+2^1+2^0 = 7$

n nodes
$n = 2^{h+1} - 1$
$n+1 = 2^{h+1}$
$\log_2(n+1) = \log_2(2^{h+1})$
$\log_2(n+1) = h+1$

max height = n-1
min height = $\lfloor \log_2 n \rfloor$

⌊ ⌋ - floor func. or gif (greatest integer smaller or equal to the no.)

$1+2+2^2+2^3+\cdots+2^k = 2^{k+1}-1$

$S = a + ar + ar^2 + \cdots + ar^{n-2} + ar^{n-1}$
$rS = ar + ar^2 + \cdots + ar^{n-2} + ar^{n-1} + ar^n$

$S - rS = a - ar^n$
$S = \frac{a(1-r^n)}{(1-r)} = \frac{a(r^n-1)}{(r-1)}$

2) What is the time complexity of the following function?

```
1  def fun(L):
2      n = len(L)
3      if n <= 1:
4          return L
5      mid = n // 2
6      left = fun(L[:mid])
7      right = fun(L[mid:])
8      for i in range(len(L)):
9          print(L[i])
10     return L
```

$R(n) = 2R(n/2) + O(n)$ $a=2$ $b=2$ $f(n) = O(n)$
$R(n) = aR(n/b) + f(n)$

TREE
fun(L) n
fun(L/2) fun(L/2) n
fun(L/4) " " " n
n

$f(n) = O(n^{\log_2 2}) = O(n) = O(n)$

$O(n^{\log_2 2} \times \log_2(n))$
$O(n \times \log n)$

$R(n) = aR(n/b) + f(n)$

1st
$f(n) = \Theta(n^{[\log_b a - e]})$ where $e>0$

$R(n) = 2R(n/2) + O(1)$

1st $f(n) = \Theta(n^{[\log_b a - e]})$ where $e > 0$

$O(n^{\log_b a})$

2nd $f(n) = \Theta(n^{\log_b a})$

$O(n^{\log_b a} \times \log_b n)$

3rd $f(n) = \Theta(n^{\lceil \log_b a + e \rceil})$ where $e > 0$

$O(f(n))$

$R(n) = 2R(n/2) + O(1)$

↳ T.C $\Rightarrow O(n)$

$R(n) = 2R(n/2) + O(n)$

↳ T.C $\Rightarrow O(n \cdot \log n)$

$R(n) = 2R(n/2) + O(n^2)$

↳ T.C $\Rightarrow O(n^2)$

$2R(N/1) + O(n)$ $n^1$

$\log_1 2$ $n^{\log_1 2}$